Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа №4

на тему:

«Позиционные игры. Метод обратной индукции»

Вариант 1

Студент: ИУ8-105 (Группа) В.В. Гоза (И. О. Фамилия)

Преподаватель: Н. С. Коннова (И.О. Фамилия)

Москва, 2026 г.

Цель работы:

Изучить метод обратной индукции и его применение к решению конечных позиционных игр с полной информацией. Изучить свойства решений таких игр.

Постановка задачи:

Найти решение конечношаговой позиционной игры с полной информацией. Для этого сгенерировать и построить дерево случайной игры согласно варианту, используя метод обратной индукции, найти решение игры и путь (все пути, если он не единственный) к этому решению. Обозначить их на дереве. Требуется выполнить генерацию случайного дерева игры заданной глубины со случайными значениями выигрышей в терминальных вершинах.

1	5	3	2, 3, 2	[–50, 50]

Рисунок 1 – условие задачи для варианта 1

Ход работы:

По условию задачи (см. рисунок 1) необходимо сгенерировать случайное дерево игры с параметрами: глубина – 5; для 3 игроков; количество стратегий для игроков – [2,3,2] соответственно; диапазон выигрышей – [-50, 50]. Затем, с помощью метода обратной индукции найти решение игры и путь к этому решению.

На рисунке 2 изображено полученное с помощью программы дерево игры, в котором зелёным цветом выделены те ребра, которые составляют путь решения игры. Оптимальный путь решения игры расписан на рисунке 3 и изображён на рисунке 4.

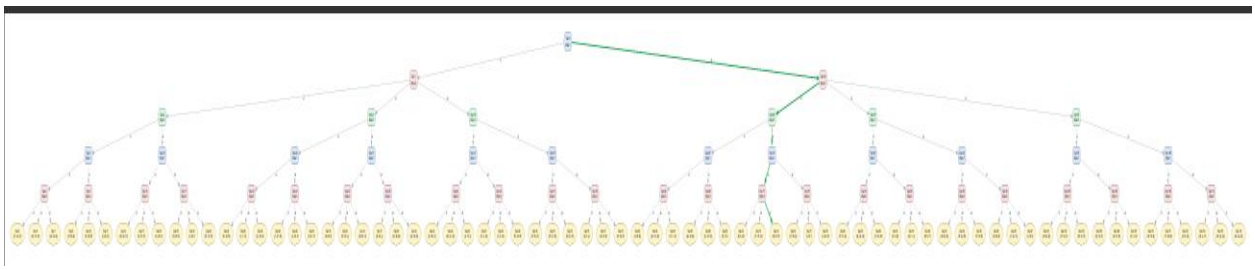

Рисунок 2 – Сгенерированное дерево игры

```
Путь 1:
  Шаг 1: узел 0 | игрок 1 | выбор s2 | переход в узел 59 | выигрыш подигры (45, 21, 18)
  Шаг 2: узел 59 | игрок 2 | выбор s1 | переход в узел 60 | выигрыш подигры (45, 21, 18)
  Шаг 3: узел 60 | игрок 3 | выбор s2 | переход в узел 70 | выигрыш подигры (45, 21, 18)
  Шаг 4: узел 70 | игрок 1 | выбор s1 | переход в узел 71 | выигрыш подигры (45, 21, 18)
  Шаг 5: узел 71 | игрок 2 | выбор s3 | переход в узел 74 | выигрыш подигры (45, 21, 18)
  Итоговый выигрыш: (45, 21, 18)
```

Рисунок 3 – Оптимальный путь решения игры

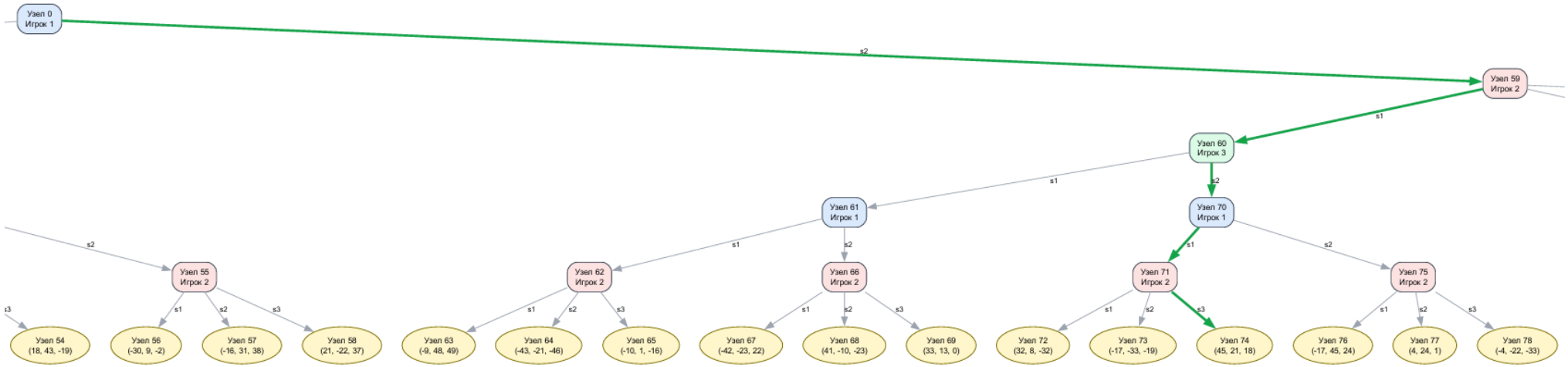

Рисунок 4 – Оптимальный путь для сгенерированного дерева

Вывод:

В результате выполнения лабораторной работы был изучен метод обратной индукции и его применение к решению конечных позиционных игр с полной информацией. Результатом работы стало сгенерированное случайное дерево игры глубины 5 со случайными значениями выигрышей в терминальных вершинах. Решение конечной позиционной игры с полной информацией было найдено с помощью метода обратной индукции. Оптимальный путь был выделен зелёным цветом на рисунке 2. Код программы, с помощью которой проводились вычисления представлен в Приложении А.

Приложение А

Листинг (также приведён по ссылке https://github.com/NeAlien/Game_theory_10_sem):

```
from __future__ import annotations

import random
from dataclasses import dataclass, field
from pathlib import Path
from typing import Optional

try:
    from graphviz import Digraph
except Exception:
    Digraph = None


SEED = 42

VARIANT_NUMBER = 1
MAX_DEPTH = 5
PLAYERS_COUNT = 3
STRATEGIES_PER_PLAYER = (2, 3, 2)
PAYOFF_RANGE = (-50, 50)

OUTPUT_DIR = Path.cwd()
TREE_BASENAME = "tree"


def payoff_to_str(payoff: tuple[int, ...]) -> str:
    return "(" + ", ".join(str(x) for x in payoff) + ")"


def payoff_list_to_str(payoffs: list[tuple[int, ...]]) -> str:
    return "[" + ", ".join(payoff_to_str(p) for p in payoffs) + "]"
```

```
def player_for_depth(depth: int) -> Optional[int]:
    if depth == MAX_DEPTH:
        return None
    return depth % PLAYERS_COUNT + 1


def strategy_label(index: int) -> str:
    return f"s{index + 1}"


@dataclass
class Node:
    idx: int
    depth: int
    player: Optional[int]
    children: list["Node"] = field(default_factory=list)
    edge_labels: list[str] = field(default_factory=list)
    payoff: Optional[tuple[int, ...]] = None
    optimal_choices: list[tuple["Node", tuple[int, ...], str]] = field(default_factory=list)
    optimal_payoffs: list[tuple[int, ...]] = field(default_factory=list)

    @property
    def is_leaf(self) -> bool:
        return self.payoff is not None


class TreeBuilder:
    def __init__(self) -> None:
        self.counter = 0

    def build(self, depth: int = 0) -> Node:
        idx = self.counter
        self.counter += 1
```

```
        node = Node(
            idx=idx,
            depth=depth,
            player=player_for_depth(depth),
        )

        if depth == MAX_DEPTH:
            node.payoff = tuple(
                random.randint(PAYOFF_RANGE[0], PAYOFF_RANGE[1])
                for _ in range(PLAYERS_COUNT)
            )
            return node

        branching = STRATEGIES_PER_PLAYER[node.player - 1]
        for i in range(branching):
            child = self.build(depth + 1)
            node.children.append(child)
            node.edge_labels.append(strategy_label(i))

        return node


def backward_induction(node: Node) -> None:
    if node.is_leaf:
        node.optimal_payoffs = [node.payoff]
        return

    for child in node.children:
        backward_induction(child)

    candidates: list[tuple[Node, tuple[int, ...], str]] = []
    for child, edge_label in zip(node.children, node.edge_labels):
        for payoff in child.optimal_payoffs:
            candidates.append((child, payoff, edge_label))
```

```
    current_player_idx = node.player - 1
    best_value = max(payoff[current_player_idx] for _, payoff, _ in candidates)

    node.optimal_choices = [
        (child, payoff, edge_label)
        for child, payoff, edge_label in candidates
        if payoff[current_player_idx] == best_value
    ]

    unique_payoffs: list[tuple[int, ...]] = []
    for _, payoff, _ in node.optimal_choices:
        if payoff not in unique_payoffs:
            unique_payoffs.append(payoff)
    node.optimal_payoffs = unique_payoffs


def collect_optimal_paths(
    node: Node,
    current_path: list[dict],
    result_paths: list[dict],
) -> None:
    if node.is_leaf:
        result_paths.append(
            {
                "steps": current_path.copy(),
                "final_payoff": node.payoff,
            }
        )
        return

    for child, payoff, edge_label in node.optimal_choices:
        current_path.append(
            {
                "from_node": node.idx,
                "player": node.player,
```

```
            "strategy": edge_label,
            "to_node": child.idx,
            "payoff_after_choice": payoff,
        }
    )
    collect_optimal_paths(child, current_path, result_paths)
    current_path.pop()


def count_nodes(node: Node) -> int:
    return 1 + sum(count_nodes(child) for child in node.children)


def count_leaves(node: Node) -> int:
    if node.is_leaf:
        return 1
    return sum(count_leaves(child) for child in node.children)


def level_nodes(root: Node) -> dict[int, list[Node]]:
    levels: dict[int, list[Node]] = {}
    stack = [root]
    while stack:
        node = stack.pop()
        levels.setdefault(node.depth, []).append(node)
        stack.extend(reversed(node.children))
    for depth in levels:
        levels[depth].sort(key=lambda x: x.idx)
    return levels


def build_optimal_edge_usage(paths: list[dict]) -> dict[tuple[int, int], int]:
    usage: dict[tuple[int, int], int] = {}
    for path in paths:
        for step in path["steps"]:
```

```
        edge = (step["from_node"], step["to_node"])
        usage[edge] = usage.get(edge, 0) + 1
    return usage


def render_tree_pdf(root: Node, paths: list[dict], output_dir: Path, basename: str) ->
Optional[Path]:
    if Digraph is None:
        print("\n[Визуализация] graphviz не установлен. PDF-дерево не построено.")
        return None

    output_dir.mkdir(parents=True, exist_ok=True)
    edge_usage = build_optimal_edge_usage(paths)

    dot = Digraph(name="LR6", format="pdf")
    dot.attr(rankdir="TB", splines="polyline",  nodesep="0.35", ranksep="0.45")
    dot.attr("node", fontname="Arial", fontsize="10")
    dot.attr("edge", fontname="Arial", fontsize="9")

    player_colors = {
        1: "#dbeafe",
        2: "#fee2e2",
        3: "#dcfce7",
    }

    def add_node(node: Node) -> None:
        node_id = f"n{node.idx}"

        if node.is_leaf:
            label = f"Узел {node.idx}\n{payoff_to_str(node.payoff)}"
            dot.node(
                node_id,
                label=label,
                shape="ellipse",
                style="filled",
```

```
            fillcolor="#fff7cc",
            color="#7c6f00",
        )
    else:
        label = f"Узел {node.idx}\nИгрок {node.player}"
        dot.node(
            node_id,
            label=label,
            shape="box",
            style="rounded,filled",
            fillcolor=player_colors.get(node.player,  "#f3f4f6"),
            color="#374151",
        )

    for child, edge_label in zip(node.children, node.edge_labels):
        child_id = f"n{child.idx}"
        usage = edge_usage.get((node.idx, child.idx), 0)

        if usage > 0:
            color = "#16a34a" if usage == 1 else "#2563eb"
            penwidth = "3.0" if usage == 1 else "4.2"
        else:
            color = "#9ca3af"
            penwidth = "1.0"

        dot.edge(
            node_id,
            child_id,
            label=edge_label,
            color=color,
            penwidth=penwidth,
        )
        add_node(child)

add_node(root)
```

```
    output_path = dot.render(filename=basename, directory=str(output_dir), cleanup=True)
    return Path(output_path)


def print_tree_stats(root: Node) -> None:
    print("\nСТАТИСТИКА СГЕНЕРИРОВАННОГО ДЕРЕВА\n")
    print(f"Всего узлов: {count_nodes(root)}")
    print(f"Терминальных вершин: {count_leaves(root)}")

    levels = level_nodes(root)
    for depth in sorted(levels):
        if depth == MAX_DEPTH:
            info = "терминальные вершины"
        else:
            info = f"ход игрока {player_for_depth(depth)}"
        print(f"Глубина {depth}: {len(levels[depth])} узл. ({info})")


def print_solution(root: Node, paths: list[dict]) -> None:
    print("\nРЕШЕНИЕ ИГРЫ\n")
    print(f"Цена игры в корне: {payoff_list_to_str(root.optimal_payoffs)}")
    print(f"Количество оптимальных путей: {len(paths)}")

    for path_number, path in enumerate(paths, start=1):
        print(f"\nПуть {path_number}:")
        for step_number, step in enumerate(path["steps"], start=1):
            print(
                f"  Шаг {step_number}: узел {step['from_node']} | "
                f"игрок {step['player']} | выбор {step['strategy']} | "
                f"переход в узел {step['to_node']} | "
                f"выигрыш подигры {payoff_to_str(step['payoff_after_choice'])}"
            )
        print(f"  Итоговый выигрыш: {payoff_to_str(path['final_payoff'])}")
```

```
def main() -> None:
    random.seed(SEED)

    builder = TreeBuilder()
    root = builder.build()

    print_tree_stats(root)
    backward_induction(root)

    optimal_paths: list[dict] = []
    collect_optimal_paths(root, [], optimal_paths)

    print_solution(root, optimal_paths)

    pdf_path = render_tree_pdf(root, optimal_paths, OUTPUT_DIR, TREE_BASENAME)
    if pdf_path is not None:
        print(f"\nPDF с деревом сохранен: {pdf_path}")


if __name__ == "__main__":
    main()
```